

LoRA Study Notes

Low-Rank Adaptation of Large Language Models

From Theory to Implementation

Everything you need to understand, use, and tune LoRA

Based on the paper: LoRA: Low-Rank Adaptation of Large Language Models
(Hu et al., 2021)

Table of Contents

1. What is LoRA? (The Big Picture)
2. The Math Behind LoRA (Simplified)
3. Why LoRA Works (Intuition)
4. LoRA vs Other Fine-Tuning Methods
5. Key Parameters Explained
6. Implementation with HuggingFace PEFT
7. Implementation with Unsloth (2x Faster)
8. QLoRA: LoRA + 4-bit Quantization
9. Which Layers to Target
10. Hyperparameter Tuning Guide
11. Common Mistakes & Fixes
12. LoRA Variants & Advanced Techniques
13. Quick Reference Cheat Sheet

1. What is LoRA? (The Big Picture)

LoRA (Low-Rank Adaptation) is a technique for fine-tuning large language models by training only a tiny fraction of the parameters. Instead of updating all the weights in a model (billions of parameters), LoRA freezes the original weights and injects small trainable matrices alongside them.

The Core Idea in One Sentence

Instead of modifying a huge weight matrix W directly, LoRA adds a small low-rank update: $W' = W + BA$, where B and A are much smaller matrices.

Why This Matters

	Full Fine-Tuning	LoRA
Parameters trained	All (e.g., 7 billion)	~0.1-2% (e.g., 4-80 million)
GPU memory needed	Very high (60+ GB for 7B)	Much lower (8-16 GB for 7B)
Training speed	Slow	2-10x faster
Storage per model	Full copy (~14 GB for 7B)	Small adapter (~10-100 MB)
Quality	Best possible	Near-identical for most tasks
Multiple tasks	One full model per task	One base + tiny adapters

Key Insight: A 7B parameter model fine-tuned with LoRA might only train 4 million parameters (0.06%), yet achieve 95-100% of full fine-tuning quality.

2. The Math Behind LoRA (Simplified)

Standard Fine-Tuning

In a neural network, each layer has a weight matrix **W** of size (d x d). During standard fine-tuning, we update W directly:

$$W_{\text{new}} = W_{\text{original}} + \Delta W$$

Where ΔW is the full-rank update matrix.

Size of ΔW = d x d (same as W, millions of parameters)

LoRA's Trick: Low-Rank Decomposition

LoRA replaces the large ΔW with two small matrices multiplied together:

Instead of:	ΔW	(d x d)	e.g., 4096 x 4096 = 16.7M params
LoRA uses:	B x A		
	B is (d x r)		e.g., 4096 x 16 = 65K params
	A is (r x d)		e.g., 16 x 4096 = 65K params
		Total:	131K params (0.8% of original!)

Where r = rank (typically 4, 8, 16, 32, or 64)

The output of a LoRA layer:

$$h = W @ x + (B @ A) @ x * (\alpha / r)$$

frozen trainable (LoRA adapter)

Initialization

- **Matrix A** is initialized with random Gaussian values
- **Matrix B** is initialized to zeros
- This means **BA = 0** at the start, so the model begins identical to the original

Why zeros for B? At the start of training, the LoRA adapter has zero effect ($BA = 0$). The model starts from its pre-trained state and gradually learns the task-specific adaptation. This is much more stable than random initialization.

3. Why LoRA Works (Intuition)

The Low-Rank Hypothesis

Research shows that the weight changes during fine-tuning live in a **low-dimensional subspace**. In other words, even though W is huge, the actual change needed (ΔW) has low rank - it can be represented by a much smaller matrix without losing important information.

Analogy: The Adjustment Knobs

Think of a pre-trained model as a complex machine with millions of knobs. Full fine-tuning adjusts ALL knobs. LoRA discovers that you only need to adjust a few key knobs (the low-rank subspace) to adapt the machine for a new task. The rest of the knobs are already set correctly from pre-training.

Key Research Findings

- The intrinsic dimensionality of fine-tuning is very low (Aghajanyan et al., 2020)
- A rank of $r=4$ to $r=16$ captures most task-specific information
- LoRA matches full fine-tuning on GLUE, GPT-2, GPT-3 benchmarks
- Higher ranks show diminishing returns ($r=64$ is rarely better than $r=16$)

When LoRA Might Not Be Enough

- Extreme domain shift (medical jargon from a general model)
- Learning a completely new language the model has never seen
- Tasks requiring fundamentally new capabilities (not just style/format)

Note: Even in these cases, LoRA often gets 90%+ of full fine-tuning quality. The gap narrows further with higher rank and more training data.

4. LoRA vs Other Fine-Tuning Methods

Method	What It Does	Params Trained	Memory	Quality
Full Fine-Tuning	Updates all weights	100%	Very High	Best
LoRA	Adds low-rank matrices to attention layers	0.1-2%	Low	Near-best
QLoRA	LoRA + 4-bit quantization	0.1-2%	Very Low	Near-best
Prefix Tuning	Prepends learned virtual tokens	0.1%	Low	Good
Prompt Tuning	Learns soft prompt embeddings	0.01%	Very Low	Moderate
Adapters	Inserts small networks between layers	1-5%	Low	Good
BitFit	Only trains bias terms	0.05%	Very Low	Moderate

Tip: LoRA is the most popular PEFT method because it offers the best quality-to-efficiency ratio. QLoRA extends it further for even lower memory. Start with QLoRA unless you have abundant GPU memory.

LoRA Advantages Over Other Methods

- **No inference latency** - adapters can be merged back into the base model
- **Composable** - swap different LoRA adapters on the same base model
- **Works on any linear layer** - attention, feedforward, embeddings
- **Battle-tested** - used in production by major companies

5. Key Parameters Explained

r (Rank) - The Most Important Parameter

The rank **r** controls the size of the LoRA matrices and directly determines how much capacity the adapter has to learn new information.

Rank (r)	Params Added	Use Case	Quality
4	Very few (~33K)	Simple style/format changes	Good
8	Few (~65K)	Single-task adaptation	Good
16	Moderate (~131K)	General fine-tuning (recommended)	Very Good
32	More (~262K)	Complex tasks, multiple skills	Excellent
64	Many (~524K)	Near full fine-tuning capacity	Excellent
128	High (~1M)	Rarely needed, diminishing returns	Marginal gain

Rule of Thumb: Start with $r=16$. Only increase if you see the model isn't learning enough (high training loss). Decrease to $r=8$ if you need to save memory.

lora_alpha - Scaling Factor

Controls how much the LoRA adapter's output is scaled before being added to the original layer output. The effective scaling is: α / r

- **$\alpha = r$** (e.g., $\alpha=16$, $r=16$): scaling = 1.0 (standard, recommended)
- **$\alpha = 2*r$** (e.g., $\alpha=32$, $r=16$): scaling = 2.0 (stronger adaptation)
- **$\alpha < r$** (e.g., $\alpha=8$, $r=16$): scaling = 0.5 (gentler adaptation)

Tip: Set $\alpha = r$ as your default. This gives a scaling factor of 1.0 and works well in most cases. Only change it if you're doing advanced tuning.

lora_dropout

- **0.0** (recommended for Unsloth and most cases)
- **0.05-0.1** for small datasets where overfitting is a risk
- Standard dropout applied to the LoRA layers during training

6. Implementation with HuggingFace PEFT

Installation

```
pip install peft transformers torch accelerate
```

Step 1: Load a Pre-trained Model

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "meta-llama/Llama-3.1-8B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype="auto",
    device_map="auto", # Automatically use available GPUs
)
```

Step 2: Configure LoRA

```
from peft import LoraConfig, get_peft_model, TaskType

lora_config = LoraConfig(
    r=16, # Rank
    lora_alpha=16, # Scaling (alpha/r = 1.0)
    lora_dropout=0.0, # No dropout
    target_modules=[ # Which layers to adapt
        "q_proj", # Query projection
        "k_proj", # Key projection
        "v_proj", # Value projection
        "o_proj", # Output projection
        "gate_proj", # FFN gate
        "up_proj", # FFN up
        "down_proj", # FFN down
    ],
    task_type=TaskType.CAUSAL_LM,
    bias="none",
)

model = get_peft_model(model, lora_config)
model.print_trainable_parameters()
# Output: trainable params: 4,194,304 || all params: 8,030,261,248
# || trainable%: 0.0522
```

Step 3: Train

```
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
```



```

tokenizer=tokenizer,
train_dataset=dataset,
args=TrainingArguments(
    output_dir="./lora_output",
    num_train_epochs=3,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    learning_rate=2e-4,
    bf16=True,
    logging_steps=10,
),
max_seq_length=2048,
)
trainer.train()

```

Step 4: Save and Load the Adapter

```

# Save only the LoRA adapter (small file, ~10-100 MB)
model.save_pretrained("./lora_adapter")

# Later: load the adapter on top of the base model
from peft import PeftModel

base_model = AutoModelForCausalLM.from_pretrained(model_name)
model = PeftModel.from_pretrained(base_model, "./lora_adapter")

# Optional: merge adapter into base (no adapter overhead at inference)
merged_model = model.merge_and_unload()
merged_model.save_pretrained("./merged_model")

```

7. Implementation with Unsloth (2x Faster)

Unsloth patches the model for optimized training. Same LoRA, 2x faster, 60% less memory. Recommended for most use cases.

```
from unsloth import FastLanguageModel

# 1. Load (automatically downloads 4-bit quantized version)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="unsloth/Meta-Llama-3.1-8B-Instruct-bnb-4bit",
    max_seq_length=2048,
    load_in_4bit=True,
)

# 2. Add LoRA adapters (Unsloth's optimized version)
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    lora_alpha=16,
    lora_dropout=0.0,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    use_gradient_checkpointing="unsloth", # 60% less VRAM
)

# 3. Train (same SFTTrainer as HuggingFace)
from trl import SFTTrainer
from transformers import TrainingArguments

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    args=TrainingArguments(
        output_dir="./output",
        num_train_epochs=3,
        per_device_train_batch_size=2,
        gradient_accumulation_steps=4,
        learning_rate=2e-4,
        bf16=True,
    ),
    max_seq_length=2048,
)
trainer.train()

# 4. Save adapter
model.save_pretrained("./output")

# 5. Export to GGUF for Ollama
model.save_pretrained_gguf(
    "./output", tokenizer,
    quantization_method="q4_k_m",
)
```

Tip: Unsloth is a drop-in replacement. Same PEFT/LoRA concepts, same SFTTrainer, just faster. Use it unless you have a specific reason not to.

8. QLoRA: LoRA + 4-bit Quantization

QLoRA (Dettrmers et al., 2023) combines LoRA with 4-bit quantization of the base model. The base weights are stored in 4-bit (NF4 format), while LoRA adapters remain in full precision (bf16/fp16).

How QLoRA Reduces Memory

Component	Full Fine-Tuning	LoRA (fp16)	QLoRA (4-bit)
Base model weights	~14 GB (fp16)	~14 GB (fp16)	~3.5 GB (4-bit)
LoRA adapters	N/A	~100 MB	~100 MB
Optimizer states	~28 GB	~200 MB	~200 MB
Gradients	~14 GB	~100 MB	~100 MB
Total for 7B model	~56 GB	~15 GB	~4 GB

QLoRA Key Innovations

- **NF4 (NormalFloat4)** - a 4-bit data type optimized for normally-distributed weights
- **Double quantization** - quantizes the quantization constants to save more memory
- **Paged optimizers** - uses CPU RAM for optimizer states when GPU runs out

QLoRA Implementation

```
from transformers import BitsAndBytesConfig
import torch

# 4-bit quantization config
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",          # NormalFloat4
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True,    # Double quantization
)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    quantization_config=bnb_config,
    device_map="auto",
)

# Then apply LoRA as normal
model = get_peft_model(model, lora_config)
```

Note: When using Unsloth with `load_in_4bit=True`, QLoRA is applied automatically. No need for manual BitsAndBytes config.

9. Which Layers to Target

LoRA can be applied to any linear layer. The choice of which layers to target affects both quality and efficiency.

Transformer Layer Architecture

```
Transformer Block:
|
|-- Multi-Head Attention:
|   |-- q_proj (Query)      &lt;-- LoRA target (always include)
|   |-- k_proj (Key)       &lt;-- LoRA target (always include)
|   |-- v_proj (Value)     &lt;-- LoRA target (always include)
|   |-- o_proj (Output)    &lt;-- LoRA target (recommended)
|
|-- Feed-Forward Network (MLP):
|   |-- gate_proj          &lt;-- LoRA target (recommended)
|   |-- up_proj            &lt;-- LoRA target (recommended)
|   |-- down_proj          &lt;-- LoRA target (recommended)
|
|-- Layer Norm (not targeted by LoRA)
```

Targeting Strategies

Strategy	Target Modules	Params	Quality	When to Use
Minimal	q_proj, v_proj	Fewest	Good	Quick experiments, very low memory
Attention Only	q_proj, k_proj, v_proj, o_proj	Low	Very Good	Standard tasks, good default
Full (Recommended)	q,k,v,o_proj + gate,up,down_proj	Moderate	Best	Production fine-tuning
+ Embeddings	All above + embed_tokens, lm_head	Most	Best+	New vocabulary, new languages

Model-Specific Module Names

Model	Attention	FFN
Llama 3 / 3.1 / 3.2	q_proj, k_proj, v_proj, o_proj	gate_proj, up_proj, down_proj
Mistral / Mixtral	q_proj, k_proj, v_proj, o_proj	gate_proj, up_proj, down_proj
Gemma 2	q_proj, k_proj, v_proj, o_proj	gate_proj, up_proj, down_proj

Phi-3	qkv_proj, o_proj	gate_up_proj, down_proj
Qwen 2.5	q_proj, k_proj, v_proj, o_proj	gate_proj, up_proj, down_proj
GPT-2 / GPT-NeoX	q_proj, k_proj, v_proj, out_proj	fc_in, fc_out

10. Hyperparameter Tuning Guide

Recommended Starting Configuration

```
# Start here for most tasks
r = 16
lora_alpha = 16          # alpha/r = 1.0
lora_dropout = 0.0
learning_rate = 2e-4
num_epochs = 3
batch_size = 2
gradient_accumulation = 4 # Effective batch = 8
max_seq_length = 2048
warmup_steps = 5
weight_decay = 0.01
```

Tuning Decision Tree

Symptom	Diagnosis	Fix
Loss doesn't decrease	Learning rate too low or rank too low	Increase lr to 5e-4 or increase r to 32
Loss spikes/oscillates	Learning rate too high	Decrease lr to 1e-4 or 5e-5
Loss goes to ~0 quickly but output is bad	Overfitting	Reduce epochs (1-2), add more data, or add dropout 0.05
Slow training	Batch too small or sequence too long	Increase batch size, reduce max_seq_length
Out of memory (OOM)	Model too large for your GPU	Reduce batch to 1, reduce seq_length, use smaller model
Output style is wrong but content is OK	Rank is fine, need more data	Add more examples that show desired style

Dataset Size Guidelines

Dataset Size	Epochs	Learning Rate	Expected Quality
< 100 examples	5-10	1e-4	Style transfer only
100-500 examples	3-5	2e-4	Good for specific tasks
500-2000 examples	2-3	2e-4	Very good quality

2000-10000 examples	1-2	1e-4	Excellent quality
> 10000 examples	1	5e-5 to 1e-4	Near full fine-tune

11. Common Mistakes & Fixes

Setting rank too high (r=256)

Problem: Wastes memory, doesn't improve quality, may overfit.

Fix: Use `r=16` as default. Only go to 32-64 if `r=16` clearly underfits.

Mismatched alpha and r

Problem: `alpha=32` with `r=4` means `scaling=8`, which causes unstable training.

Fix: Set `alpha = r` for scaling of 1.0, or `alpha = 2*r` for slightly stronger.

Not using chat template

Problem: Model learns wrong format, outputs garbled text.

Fix: Always use `tokenizer.apply_chat_template()` to format training data.

Training on too-short sequences

Problem: Model truncates long inputs at inference.

Fix: Set `max_seq_length` to cover your longest expected input + output.

Forgetting to merge before export

Problem: Deploying the adapter separately adds inference latency.

Fix: Use `model.merge_and_unload()` before exporting to GGUF.

Using fp16 on non-Ampere GPUs

Problem: Older GPUs (V100, T4) don't support bf16.

Fix: Use `fp16=True` instead of `bf16=True` on pre-Ampere GPUs.

Not freezing the base model

Problem: If using raw PEFT without Unsloth, base may still be trainable.

Fix: LoRA automatically freezes base weights. Verify with `print_trainable_parameters()`.

Training too many epochs on small data

Problem: Model memorizes training data, outputs exact copies.

Fix: Use 1-3 epochs for small datasets. Watch for loss approaching 0.

12. LoRA Variants & Advanced Techniques

Variant	Key Idea	When to Use
LoRA (original)	Low-rank decomposition of weight updates	Default choice for most tasks
QLoRA	LoRA + 4-bit quantized base model	Limited GPU memory (most common)
LoRA+	Different learning rates for A and B matrices	Slightly better quality at same cost
DoRA	Decomposes into magnitude + direction components	Better quality on some benchmarks
rsLoRA	Scales alpha by $1/\sqrt{r}$ instead of $1/r$	When using very high ranks ($r > 64$)
AdaLoRA	Adaptively adjusts rank per layer during training	When some layers need more capacity
LoRA-FA	Freezes A after init, only trains B	Faster training, similar quality

Multi-Adapter / Adapter Composition

One of LoRA's biggest advantages is that you can train multiple adapters for different tasks and swap/combine them at inference time:

```
from peft import PeftModel

# Load base model once
base = AutoModelForCausalLM.from_pretrained("meta-llama/...")

# Load different adapters for different tasks
coding_model = PeftModel.from_pretrained(base, "./coding_adapter")
medical_model = PeftModel.from_pretrained(base, "./medical_adapter")
legal_model = PeftModel.from_pretrained(base, "./legal_adapter")

# Each adapter is only 10-100 MB!
# The base model (14 GB) is shared across all tasks
```

Key Insight: This is LoRA's killer feature for production. One base model in GPU memory, with tiny adapters hot-swapped per request. This is how companies serve hundreds of custom models efficiently.

13. Quick Reference Cheat Sheet

Default Configuration (Copy-Paste Ready)

```
# LoRA Config - works for 90% of use cases
lora_config = LoraConfig(
    r=16,
    lora_alpha=16,
    lora_dropout=0.0,
    target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
                    "gate_proj", "up_proj", "down_proj"],
    task_type=TaskType.CAUSAL_LM,
    bias="none",
)
```

Quick Decision Guide

Question	Answer
What rank should I use?	Start with 16. Go to 32 if underfitting.
What alpha should I set?	Same as r (e.g., r=16, alpha=16).
Which modules to target?	All 7: q,k,v,o_proj + gate,up,down_proj.
How many epochs?	3 for 100-1000 examples. 1-2 for 1000+.
What learning rate?	2e-4 as default. Lower for large datasets.
LoRA or QLoRA?	QLoRA unless you have 40+ GB VRAM.
When to use full fine-tuning?	Only if LoRA quality isn't enough AND you have the GPU budget.

Memory Cheat Sheet (7B Model)

Method	VRAM Needed	Training Speed
Full Fine-Tuning (fp16)	~60 GB	Baseline
LoRA (fp16)	~16 GB	~1.5x faster
QLoRA (4-bit)	~6 GB	~1.5x faster
QLoRA + Unsloth	~5 GB	~3x faster

Key Formulas

$\text{Trainable params} = 2 * r * d * \text{num_target_modules} * \text{num_layers}$

Scaling factor = α / r
Effective batch = $\text{per_device_batch_size} * \text{gradient_accumulation_steps}$
Adapter size = $\text{trainable_params} * 2 \text{ bytes (fp16)}$ # ~MB

Example for Llama-3.1-8B (d=4096, 32 layers, 7 modules):

Trainable = $2 * 16 * 4096 * 7 * 32 = 29.4\text{M}$ params

Adapter size = $29.4\text{M} * 2 = \sim 59 \text{ MB}$

References: Hu et al. (2021) 'LoRA: Low-Rank Adaptation of Large Language Models'; Dettmers et al. (2023) 'QLoRA: Efficient Finetuning of Quantized Language Models'; Aghajanyan et al. (2020) 'Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning'